Experiment 7: Selection Sort on Random, Reverse and Sorted Arrays

Aim: To implement Selection Sort and compare its performance on random, reverse-sorted, and already-sorted arrays.

Algorithm

1. Start the program.
2. Initialize three arrays: random, reverse-sorted, and already-sorted.
3. Set the first element as the minimum element.
4. Compare it with the remaining unsorted elements to find the smallest element.
5. Swap the smallest element with the first element of the unsorted section.
6. Repeat until the entire array is sorted.
7. Record the execution time for each array.
8. Display the sorted arrays and their execution times.
9. Stop the program.

C

```
#include <stdio.h>

#include <time.h>

void selectionSort(int a[], int n) {

    int i, j, min, temp;

    for (i = 0; i < n - 1; i++) {

        min = i;

        for (j = i + 1; j < n; j++) {

            if (a[j] < a[min])

                min = j;

        }



        if (min != i) {

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;
```

```
        }
    }
}
void display(int a[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
int main() {
    int random[] = {6, 3, 8, 2, 7};
    int reverse[] = {5, 4, 3, 2, 1};
    int sorted[] = {1, 2, 3, 4, 5};
    int n = 5;
    clock_t start, end;
    double time_taken;
    printf("Random array before sorting: ");
    display(random, n);
    start = clock();
    selectionSort(random, n);
    end = clock();
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Random array after sorting: ");
    display(random, n);
    printf("Time taken: %f seconds\n\n", time_taken);
```

```
    printf("Reverse array before sorting: ");
    display(reverse, n);
    start = clock();
    selectionSort(reverse, n);
    end = clock();
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Reverse array after sorting: ");
    display(reverse, n);
    printf("Time taken: %f seconds\n\n", time_taken);
    printf("Sorted array before sorting: ");
    display(sorted, n);
    start = clock();
    selectionSort(sorted, n);
    end = clock();
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sorted array after sorting: ");
    display(sorted, n);
    printf("Time taken: %f seconds\n", time_taken);
    return 0;
}
```

Output:

```
Random array before sorting: 6 3 8 2 7
Random array after sorting: 2 3 6 7 8
Time taken: 0.000002 seconds

Reverse array before sorting: 5 4 3 2 1
Reverse array after sorting: 1 2 3 4 5
Time taken: 0.000001 seconds

Sorted array before sorting: 1 2 3 4 5
Sorted array after sorting: 1 2 3 4 5
Time taken: 0.000001 seconds
```

Result

Thus, the C program successfully implements Selection Sort and compares its performance on random, reverse-sorted, and already-sorted arrays. Selection Sort performs approximately the same number of comparisons in all three cases.